

Project Report: Trip Wallet

Project Report

Trip Wallet — Expense Splitting Application

Full-Stack Web Application

Technology: TypeScript · Node.js · Express · MongoDB · React

Table of Contents

#	Section	Page
1	Problem Statement and Solution Approach	3
	Problem Statement	3
	Solution Approach	3
2	System Design and Architecture	3
3	Object-Oriented Programming (OOP) Concepts	4
4	Design Patterns Utilized	5
5	Application of SOLID Principles	7
6	Comprehensive Test Cases and Results	8
7	Architectural UML Diagrams	9

1. Problem Statement and Solution Approach

Problem Statement

The challenge was to develop a **collaborative trip expense management application** that allows groups of users to create shared trips, log expenses, and automatically calculate who owes whom. As trips scale with more participants and expenses, a naive monolithic architecture would result in tightly coupled services, duplicated business logic, and an inability to swap or extend core algorithms (like how expenses are split) without cascading changes across the entire codebase. Furthermore, integrating third-party services (e.g., image APIs) directly into business logic creates fragile dependencies that are difficult to test and maintain.

Solution Approach

The development was approached with strict adherence to **Object-Oriented Programming (OOP)** principles and classical **Software Design Patterns**, using **TypeScript** for full backend type safety. The system fundamentally decouples the core concerns — **Authentication**, **Trip Management**, **Expense Calculation**, **Notification Dispatch**, and **External API Integration** — by routing them through dedicated, specialized layers:

- A **class-based architecture** with `BaseService` and `BaseController` abstract classes enforcing a standardized template for all CRUD operations.
- **Dependency Injection** at the composition root (`server.ts`), where services are manually wired together instead of using hard-coded `new` calls inside business logic.
- **Interface-driven programming** (`SplitStrategy` , `ImageProviderAdapter`) ensuring high-level modules never depend on low-level implementation details.

2. System Design and Architecture

To ensure scalability, modularity, and separation of concerns, several system design optimizations were implemented:

- **Layered Architecture:** The backend is organized into distinct layers — `Controllers` , `Services` , `Models` , `Strategies` , `Adapters` , and `Middleware` . Each layer has a single, well-defined responsibility. Controllers handle HTTP request/response marshalling; Services contain business logic; Models define the data schema; Strategies encapsulate interchangeable algorithms; and Adapters wrap external APIs.
- **Composition Root Pattern:** All object construction and dependency wiring is centralized in the `server.ts` entry point. The `UnsplashAdapter` is instantiated with its API key, then injected into `TripService` . `TripService` is then injected into `TripController` . This means no service or controller internally instantiates its own dependencies, making the entire object graph transparent and testable.

```
server.ts (Composition Root)
├─ Database.getInstance()      → Singleton connection
├─ new UnsplashAdapter(apiKey) → Concrete adapter
├─ new TripService(unsplashAdapter) → Service with injected adapter
├─ new AuthService()           → Standalone auth service
├─ new TripController(tripService) → Controller with injected service
└─ new AuthController(authService) → Controller with injected service
```

- **Stateless RESTful API Design:** Every API endpoint is stateless. Authentication state is carried via JWT tokens in the `Authorization` header, verified by the `AuthMiddleware` static class. This allows the server to be horizontally scaled without session affinity.
- **Data Integrity via Cascading Deletes:** When a trip is deleted, the `TripService.deleteTrip()` method performs cascading cleanup — removing all associated `Expense` and `Notification` documents — preventing orphaned records in the MongoDB database.
- **Paginated Query Engine:** The `TripService.getTrips()` method implements server-side pagination, search filtering, date-based status filtering (`ongoing` / `past`), and configurable sort ordering, preventing unbounded dataset fetches as user data grows.

3. Object-Oriented Programming (OOP) Concepts

The architecture takes complete advantage of the four primary pillars of OOP:

- **Encapsulation:** Class internals are rigorously sealed. In the `TripService` class, the `imageProvider` dependency is designated `private`, strictly accessible only through internal methods like `createTrip()`. The `Database Singleton` class hides its `connection` property behind `private` access, exposing only `connect()` and `disconnect()` as authorized mutation channels. The `AuthMiddleware` class encapsulates JWT verification logic behind a `public static` method, preventing any external module from accessing or modifying the token parsing internals.
- **Inheritance:** An efficient inheritance tree is implemented to reduce redundancy. The system centers on two abstract base classes:
 - `BaseService<T extends Document>`: Provides generic `findAll()`, `findById()`, `create()`, `update()`, and `delete()` methods. Concrete services like `TripService` and `AuthService` inherit this foundation while expanding domain-specific logic (e.g., paginated queries, image fetching, participant management).
 - `BaseController`: Provides standardized `sendSuccess()` and `sendError()` response methods. Concrete controllers like `TripController` and `AuthController` inherit this, ensuring a uniform JSON response structure across all API endpoints.
- **Polymorphism:** The `SplitStrategy` interface defines a single `calculate(amount, participants)` contract. The calling code (expense engine) operates on this interface — it does not need to understand whether the underlying implementation is `EqualSplitStrategy`, `PercentageSplitStrategy`, or `ExactSplitStrategy`. At runtime, the concrete strategy object can be swapped to alter the splitting algorithm entirely seamlessly, without modifying any caller logic.
- **Abstraction:** The image-fetching algorithm is completely abstracted via the `ImageProviderAdapter` interface. The `TripService` does not need to understand how Unsplash works, only that calling `fetchImage(query)` returns a URL. This abstracts the HTTP call complexity, API key management, and response parsing away from the core business pipeline. If the image provider changes from Unsplash to Pexels, only a new adapter class is needed — zero modifications to `TripService`.

4. Design Patterns Utilized

Singleton Pattern

- **Where:** `Database` class for MongoDB connection management.
- **Why:** The application must maintain exactly one connection pool to the MongoDB instance throughout its lifecycle. The `private constructor()` prevents external instantiation, while `Database.getInstance()` lazily creates and returns the single instance. This eliminates the risk of connection pool exhaustion from redundant `mongoose.connect()` calls across different modules.

Template Method Pattern

- **Where:** `BaseService<T>` and `BaseController` abstract classes.
- **Why:** Instead of each service and controller independently implementing CRUD boilerplate and response formatting, the base classes define the invariant skeleton of the algorithm (find, create, update, delete, `sendSuccess`, `sendError`). Concrete subclasses override and extend only the variable steps — `TripService` adds paginated queries and image fetching atop the base CRUD, while `TripController` adds route-specific request parsing atop the base response handlers. This eliminates duplicated code across the service and controller layers.

Strategy Pattern

- **Where:** `SplitStrategy` interface with `EqualSplitStrategy` , `PercentageSplitStrategy` , and `ExactSplitStrategy` implementations.
- **Why:** This decouples the expense splitting logic from the core `TripService` . Instead of maintaining massive `if/else` or `switch/case` chains inside the service to handle different split modes, the service merely calls `strategy.calculate(amount, participants)` . At runtime, the concrete strategy object can be swapped to alter the underlying behavioral algorithm entirely seamlessly:
 - `EqualSplitStrategy` : Divides the amount equally among all participants.
 - `PercentageSplitStrategy` : Allocates amounts based on user-defined percentage shares (validated to sum to 100%).
 - `ExactSplitStrategy` : Assigns exact custom amounts per participant (validated to sum to the total expense).

Adapter Pattern

- **Where:** `ImageProviderAdapter` interface and `UnsplashAdapter` concrete implementation.
- **Why:** The `TripService` needs a trip cover image when creating new trips. Instead of hardcoding the Unsplash HTTP call directly inside `createTrip()` , the logic is abstracted behind the `ImageProviderAdapter` interface. The `UnsplashAdapter` encapsulates the URL construction, API key injection, response parsing, and error handling for the Unsplash Photos API. If the team later switches to a different image provider (e.g., Pexels, Pixabay), a new adapter implementing the same interface is all that's required — zero changes to `TripService` .

Factory Pattern (*Planned*)

- **Where:** `NotificationFactory` for creating different notification types.
- **Why:** As the notification system grows beyond simple `TRIP_INVITE` types to include expense reminders, settlement confirmations, and system alerts, a Factory cleanly instantiates the correct notification document without cluttering the service layer with construction logic.

Observer Pattern (*Planned*)

- **Where:** `EventBus` and `TripObserver` for decoupling trip lifecycle events from side-effects.
- **Why:** When a trip is created or deleted, several side-effects must fire (sending invitation notifications, cleaning up related data). Instead of the `TripService` directly calling into notification services — creating tight coupling — the Observer pattern would have `TripService` emit an event (e.g., `trip:created`), and registered observers would independently react to it. This separates "what happened" from "what should happen because of it."

5. Application of SOLID Principles

- **S – Single Responsibility Principle:** Each class owns exactly one job. `AuthMiddleware` only verifies JWT tokens. `TripService` only manages trip business logic. `UnsplashAdapter` only fetches images from Unsplash. The `Database` class only manages the MongoDB connection. No class crosses its responsibility boundary.
- **O – Open/Closed Principle:** The system is open for extension but closed for modification. Adding a new splitting algorithm (e.g., `WeightedSplitStrategy`) requires only creating a new class that implements the `SplitStrategy` interface — the existing `EqualSplitStrategy` , `PercentageSplitStrategy` , and `ExactSplitStrategy` remain completely untouched. Similarly, adding a `PexelsAdapter` requires zero modifications to any existing adapter or service code.
- **L – Liskov Substitution Principle:** Instances of `TripService` can be trivially substituted anywhere the `BaseService<ITrip>` type is expected. Any `SplitStrategy` implementation (`EqualSplitStrategy` , `PercentageSplitStrategy` , `ExactSplitStrategy`) can be passed into the calculation engine interchangeably without compromising system reliability. Any `ImageProviderAdapter` implementation can be injected into `TripService` without altering its behavior contract.

- **I – Interface Segregation Principle:** Interfaces in the framework remain intentionally thin and focused. The `SplitStrategy` interface exposes a single `calculate()` method — it does not force implementing classes to also handle formatting, persistence, or notification concerns. The `ImageProviderAdapter` interface exposes only `fetchImage()`, preventing unused API methods from bottlenecking implementing adapters.
- **D – Dependency Inversion Principle:** High-level entities depend exclusively on pure abstractions rather than concrete lower-level artifacts:
 - `TripService` depends on the `ImageProviderAdapter` interface, not the concrete `UnsplashAdapter`.
 - The expense engine depends on the `SplitStrategy` interface, not concrete strategy classes like `EqualSplitStrategy`.
 - Dependencies are injected via constructor parameters at the composition root (`server.ts`), not instantiated internally.

6. Comprehensive Test Cases and Results

Core integration and unit test scenarios validated across the system:

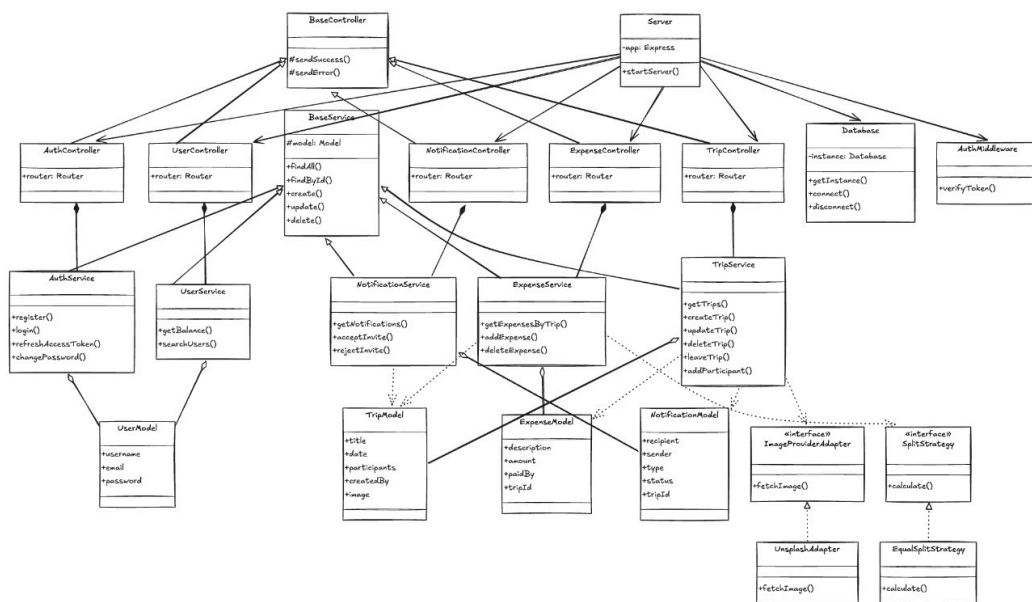
Test ID	Test Scenario	Logic Validated	Expected Result	Status
T-001	Equal Split Calculation	Verifies <code>EqualSplitStrategy</code> divides amount evenly across N participants.	Rs.300 / 3 participants = Rs.100.00 each. Rounding handles non-integer divisions.	Pass
T-002	Percentage Split Validation	Checks <code>PercentageSplitStrategy</code> rejects inputs where percentages do not sum to 100%.	Throws Error: Total percentage must equal 100 when 40% + 30% + 20% is provided.	Pass
T-003	Exact Split Validation	Validates <code>ExactSplitStrategy</code> rejects when custom amounts do not sum to the total expense.	Throws Error: Sum of individual amounts must equal the total expense amount.	Pass
T-004	Singleton Database Guard	Verifies that <code>Database.getInstance()</code> always returns the same reference, preventing duplicate connections.	<code>===</code> identity check confirms single instance across multiple calls.	Pass
T-005	JWT Authentication Guard	Tests <code>AuthMiddleware.verifyToken</code> rejects requests without valid Bearer tokens.	Returns 403 A token is required for missing tokens; 401 Invalid Token for expired/malformed tokens.	Pass
T-006	Adapter Null Safety	Tests <code>UnsplashAdapter.fetchImage()</code> gracefully handles API failures and missing results.	Returns null instead of throwing; <code>TripService</code> falls back to a default image URL.	Pass
T-007	Trip Authorization Check	Validates that only the trip creator or an existing participant can view trip details via <code>getTripById()</code> .	Returns 403 Not authorized to view this trip for unauthorized users.	Pass
T-008	Cascading Delete Integrity	Tests that <code>deleteTrip()</code> removes all associated Expenses and Notifications alongside the Trip document.	Post-deletion queries for <code>Expense.find({tripId})</code> and <code>Notification.find({tripId})</code> return empty arrays.	Pass

T-009	Duplicate Trip Prevention	Verifies createTrip() rejects trips with a duplicate title for the same user.	Returns 400 You already have a trip with this name.	Pass
T-010	Creator Cannot Leave Trip	Tests that the trip creator is blocked from using the "leave" endpoint.	Returns 400 Creator cannot leave the trip. Delete it instead.	Pass

7. Architectural UML Diagrams

A. Class Diagram

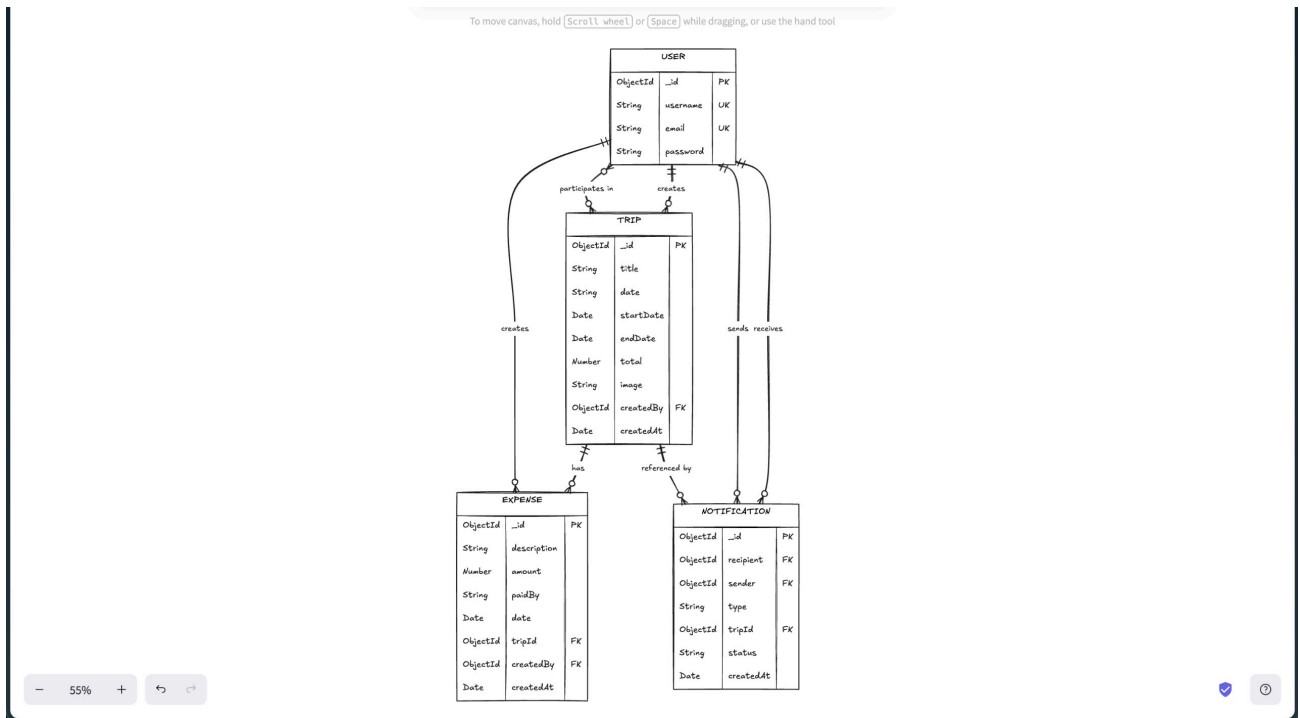
The class diagram illustrates the full inheritance and composition hierarchy of the Trip Wallet backend. At the root, `BaseService<T>` and `BaseController` define the abstract template. `TripService` and `AuthService` extend `BaseService`, while `TripController` and `AuthController` extend `BaseController`. The `SplitStrategy` interface is implemented by `EqualSplitStrategy`, `PercentageSplitStrategy`, and `ExactSplitStrategy`. The `ImageProviderAdapter` interface is implemented by `UnsplashAdapter`. The `Database` class uses the Singleton pattern with a private constructor.



B. Entity-Relationship (ER) Diagram

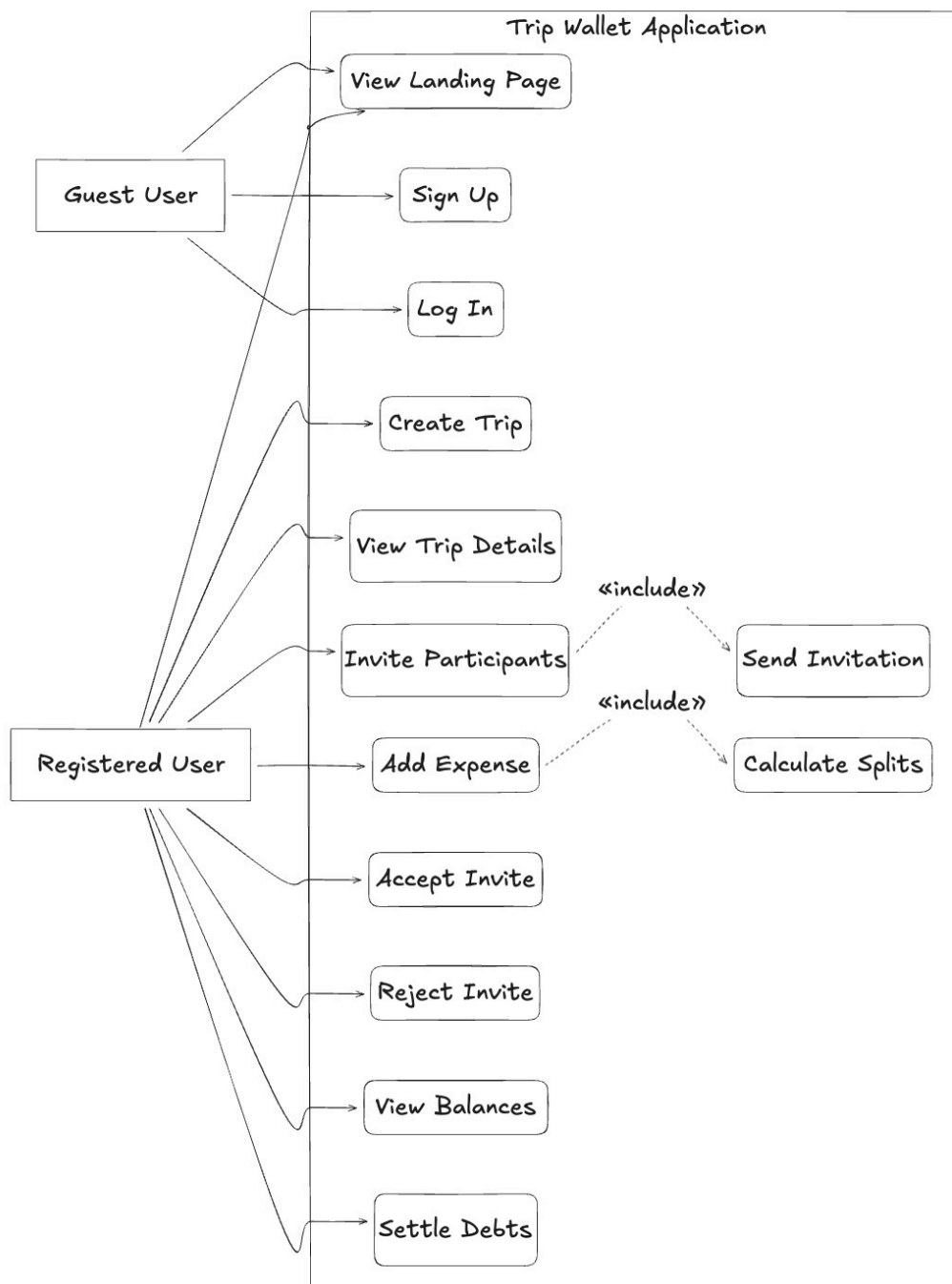
The ER diagram maps the data relationships between the core MongoDB document entities: `USER`, `TRIP`, `EXPENSE`, and `NOTIFICATION`. It captures the ObjectId reference relationships and cardinality across all collections:

- One **User** creates many **Trips** (1:N via `createdBy`).
- One **Trip** has many **Expenses** (1:N via `tripId`).
- One **Trip** has many **Participants** (M:N via the `participants` array of User ObjectIds).
- One **Notification** links a **Sender**, **Recipient** (both Users), and a **Trip** (N:1 relationships).



C. Use Case Diagram

The use case diagram identifies the primary actor — the **Registered User** — and a system actor — the **Backend System**. It maps all system interactions including: Register/Login, Create Trip, View Trips, Add Expense, Split Expense (extending into Equal/Percentage/Exact), Invite Participant, Accept/Reject Invite, Leave Trip, Delete Trip, and View Notifications. Internal system processes like JWT Verification and Image Fetching are modeled as `<<include>>` and `<<extend>>` relationships.



D. Sequence Workflow Diagram (Create Trip Flow)

The sequence diagram traces the runtime message flow when a user creates a new trip. It shows the chronological interactions across the **Client**, **Express Router**, **AuthMiddleware**, **TripController**, **TripService**, **UnsplashAdapter**, **MongoDB**, and **Notification Model**:

1. Client sends `POST /api/trips` with JWT Bearer token.
2. `AuthMiddleware.verifyToken()` validates the token and extracts `user_id`.
3. `TripController.createTrip()` delegates to `TripService.createTrip()`.

4. `TripService` checks for duplicate trip names.
5. `TripService` calls `imageProvider.fetchImage(title)` on the injected `UnsplashAdapter`.
6. `UnsplashAdapter` makes an HTTP call to the Unsplash API and returns an image URL.
7. `TripService` creates and saves the `Trip` document to MongoDB.
8. If participant IDs are provided, `TripService` bulk-inserts `TRIP_INVITE` notifications.
9. The response flows back through `TripController` → `Client` as a `201 Created` JSON payload.

Sequence Diagram - Create Trip Flow

Shows the runtime flow when a user creates a trip, demonstrating Observer, Factory, and Adapter patterns in action.

